ВЫЧИСЛИТЕЛЬНЫЕ МЕТОДЫ · ЛЕКЦИЯ 1

Python для научных вычислений

Основы языка, NumPy и визуализация данных

Лю Шисян

Кафедра теплофизики (Э6)
МГТУ им. Н.Э. Баумана
Москва / 2026

Место этой лекции в курсе



Лекция 1 — Python для научных вычислений

В этой лекции вычислим значения функций на сетке и построим графики. В конце разберём охлаждение тела: от параметров модели до температуры в заданный момент времени.

Что понадобится для расчёта:

- переменные и функции для записи формул;
- массивы NumPy для вычислений сразу во многих точках;
- Matplotlib для графика и проверки поведения решения.

План

1. Введение в Python
2. Основы языка Python
3. Научные вычисления с NumPy
4. Визуализация с Matplotlib

1

Введение в Python

Python и библиотеки для научных вычислений



Python — язык программирования.

- переменные, числа, строки и структуры данных;
- условия, циклы и функции.



NumPy — вычисления с массивами.

- создание массивов, индексация и поэлементные операции;
- векторы, матрицы и линейная алгебра.



Matplotlib — визуализация данных.

- линии, точечные графики и гистограммы;
- двумерные поля, трёхмерные поверхности и сохранение рисунков.

Среды разработки Python

VS Code

Универсальная среда для программирования.

- встроенный терминал;
- WSL и SSH;
- расширение Python.

Jupyter Notebook

Интерактивная среда в браузере.

- короткие примеры;
- пошаговые вычисления;
- лабораторные работы.

PyCharm

Полноценная IDE для Python.

- удобна для крупных проектов;
- навигация и отладка кода;
- управление окружением.

Spyder

Научная среда, похожая на MATLAB.

- редактор кода;
- интерактивная консоль;
- просмотр переменных.

В курсе: VS Code — для программ, Jupyter Notebook — для интерактивных вычислений.

Управление окружением с помощью Miniconda

Окружение для курса

Окружение содержит свою версию Python и установленные библиотеки. Имя `numcalc` выбираем сами. Команда `activate` переключает терминал на это окружение.

1. Создание окружения

```
conda create -n numcalc python=3.12
```

2. Активация окружения

```
conda activate numcalc
```

3. Установка пакетов

```
conda install numpy scipy
    matplotlib
conda install jupyter notebook
```

4. Запуск Jupyter Notebook

```
jupyter notebook
```

2

Основы языка Python

Переменные и числовые типы

Значения и типы

```
a = 5
b = 2.0
z = 3 + 4j

print(type(a))
print(type(b))
print(type(z))
```

```
<class 'int'>
<class 'float'>
<class 'complex'>
```

`j` обозначает мнимую единицу.

Арифметические операции

При `a = 5, b = 2.0`:

Выражение	Результат	Операция
`a + b`	`7.0`	Сложение
`a - b`	`3.0`	Вычитание
`a * b`	`10.0`	Умножение
`a / b`	`2.5`	Деление
`a // b`	`2.0`	Деление с округлением вниз
`a % b`	`1.0`	Остаток от деления
`a ** b`	`25.0`	Возведение в степень

Строки: `str`

```
s1 = "hello "
s2 = 'world'
s = s1 + s2
```

В конце `s1` есть пробел: он тоже считается символом.

Выражение	Результат в Notebook	Операция
`s`	`'hello world'`	Соединение строк
`s1 * 3`	`'hello hello hello '`	Повторение
`s1[0]`	`'h'`	Первый символ
`s1[0:3]`	`'hel'`	Символы с индексами 0, 1, 2
`len(s1)`	6	Длина с учётом пробела
`s.split()`	`['hello', 'world']`	Разделение на слова
`s.replace('h', 'H')`	`'Hello world'`	Замена символа

Индексы начинаются с 0. Правая граница среза не включается.

Булевы значения: bool

Логические операции

```
x = True
y = False
```

`x and y`	`False`	И
`x or y`	`True`	ИЛИ
`not x`	`False`	НЕ

Преобразование в bool

`a = 10`	`bool(a)`	`True`
`b = 0`	`bool(b)`	`False`
`c = ”`	`bool(c)`	`False`
`d = []`	`bool(d)`	`False`
`e = None`	`bool(e)`	`False`

Сравнение чисел

```
a = 2
b = 3
```

`a == b`	`False`	`a != b`	`True`
`a > b`	`False`	`a < b`	`True`

= — присваивание; == — сравнение.

Условие в программе

```
age = 20
if age >= 18:
    print(”Взрослый”)
else:
    print(”Несовершеннолетний”)
```

Вывод: Взрослый

Список: `list`

Список — изменяемая последовательность. Значения могут повторяться.

```
lst = [1, 2, 3, 4]
```

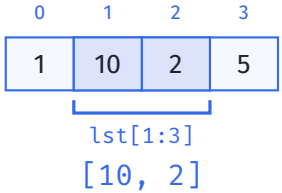
Команды выполняются по порядку

Команда	После команды
`lst.append(5)`	`[1, 2, 3, 4, 5]`
`lst.insert(1, 10)`	`[1, 10, 2, 3, 4, 5]`
`lst.remove(3)`	`[1, 10, 2, 4, 5]`
`lst.pop(3)`	`[1, 10, 2, 5]`

`pop(3)` возвращает удалённое значение 4.

`lst + 1` — `TypeError`: оператор `+` соединяет списки.

После изменений



Индексы начинаются с 0.

Правая граница среза не включается.

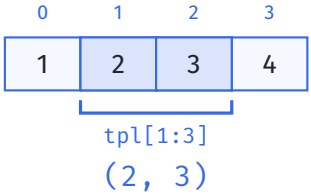
Кортеж: `tuple`

Кортеж — неизменяемая последовательность.

```
tpl = (1, 2, 3, 4)
```

Выражение	Результат
`tpl[0]`	`1`
`tpl[1:3]`	`(2, 3)`
`len(tpl)`	`4`
`3 in tpl`	`True`

Индексы и срез



Элементы нельзя заменять, добавлять или удалять.

`tpl.append(5)` — `AttributeError`: у кортежа нет метода `append()`.

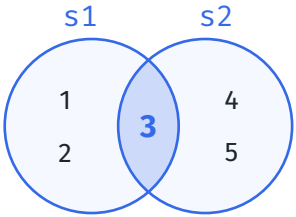
Множество: set

Множество хранит уникальные элементы.

```
s = {1, 2, 3, 4}
s.add(5)
s.remove(3)
```

После команд: `s = {1, 2, 4, 5}`.

```
s1 = {1, 2, 3}
s2 = {3, 4, 5}
```



`s1 & s2` → `{3}`

Порядок не гарантирован.
Доступа по индексу нет.

Выражение	Результат	Операция
`s1 | s2`	`{1, 2, 3, 4, 5}`	Объединение
`s1 & s2`	`{3}`	Пересечение
`s1 - s2`	`{1, 2}`	Разность

Словарь: dict

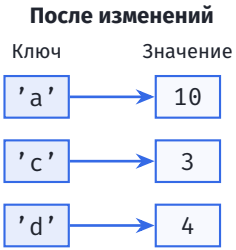
Словарь хранит пары **ключ–значение**. Ключи уникальны.

```
dic = {'a': 1, 'b': 2, 'c': 3}
```

`dic['b']` ⟶ 2.

```
dic['d'] = 4
dic['a'] = 10
del dic['b']
```

Добавляем `'d'`, меняем значение `'a'`, удаляем `'b'`.



Метод	Что возвращает
`dic.keys()`	Ключи
`dic.values()`	Значения
`dic.items()`	Пары ключ–значение

Пользовательская функция: закон Фурье

Теплопроводность плоского слоя

При линейном профиле температуры:

$$q = -k\frac{\Delta T}{L}, \qquad \Delta T = T_2 - T_1.$$

- k — теплопроводность, Вт/(м · К);
- ΔT — разность температур, К;
- L — толщина слоя, м.

q — плотность теплового потока, Вт/м^2.

Знак минус: тепло переносится в сторону меньшей температуры.

Определение и вызов функции

```
def heat_flux(k, dT, L):
    q = -k * dT / L
    return q

q = heat_flux(150, 20, 0.01)
print(q)
```

Вывод Python

```
-300000.0
```

`def` определяет функцию.
`return q` возвращает вычисленное значение.



Анонимные функции: lambda

`lambda` создаёт функцию из одного выражения.

Квадрат одного числа

```
square = lambda x: x ** 2
print(square(5))
```

Вывод: 25

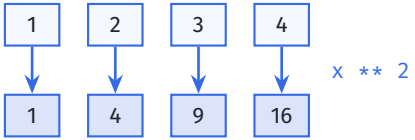


x — параметр.
Значение `x ** 2` становится результатом функции.

Квадрат каждого элемента

```
nums = [1, 2, 3, 4]
squared = list(map(
    lambda x: x ** 2, nums))
print(squared)
```

Вывод: `[1, 4, 9, 16]`



`map` применяет функцию к каждому элементу.
`list` собирает результаты в список.

Цикл `for`: перебор элементов

`for` получает элементы по одному. Каждый `print` выводит отдельную строку.

Список

```
nums = [1, 2, 3, 4]
for num in nums:
    print(num)
```

Вывод: 1 → 2 → 3 → 4

Строка

```
text = "abcd"
for char in text:
    print(char)
```

Вывод: a → b → c → d

Словарь: ключ и значение

```
dic = {'a': 1, 'b': 2, 'c': 3}
for key, value in dic.items():
    print(f"{key}: {value}")
```

Вывод: a: 1 → b: 2 → c: 3

Диапазон: `range`

```
for i in range(4):
    print(i)
```

Вывод: 0 → 1 → 2 → 3

`range(4)`: от 0 до 3; число 4 не входит.

Циклы: `while`, `break`, `continue`

`while` проверяет условие перед каждой итерацией. При `False` цикл завершается.

Счётчик с шагом 1

```
count = 0
while count < 5:
    print(count)
    count += 1
```

Вывод: 0 → 1 → 2 → 3 → 4

Уменьшение на 2

```
n = 10
while n > 0:
    print(n)
    n -= 2
```

Вывод: 10 → 8 → 6 → 4 → 2

`break`: выход из цикла

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

Вывод: 0 → 1 → 2 → 3 → 4

`continue`: следующая итерация

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

Вывод: 0 → 1 → 3 → 4

3

Научные вычисления с NumPy

Операции со списками Python

```
a = [1, 2, 3, 4]
b = [2, 3, 4, 5]
```

Оператор + соединяет списки

```
print(a + b)
```

```
[1, 2, 3, 4, 2, 3, 4, 5]
```

`a + 1` вызывает `TypeError`.

Вычисление для каждого элемента

```
print([x + 1 for x in a])
print([x + y
       for x, y in zip(a, b)])
```

```
[2, 3, 4, 5]
[3, 5, 7, 9]
```



`a + b`: восемь элементов

NumPy: поэлементные операции

Массив и число

```
import numpy as np

a = np.array([1, 2, 3, 4])
print(a + 1)
```

```
[2 3 4 5]
```

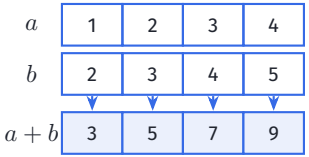
Число прибавляется к каждому элементу.

Два массива

```
b = np.array([2, 3, 4, 5])
print(a + b)
```

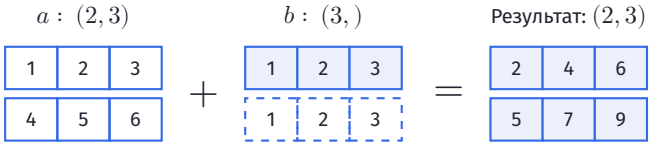
```
[3 5 7 9]
```

Складываются элементы на одинаковых позициях.



Broadcasting: прибавляем строку к матрице

```
a = np.array([[1, 2, 3], [4, 5, 6]])
b = np.array([1, 2, 3])
print(a + b)
```



Вектор b прибавляется к **каждой строке** массива a.

Размеры сравниваются справа: они должны совпадать, либо один из них равен 1.

Создание массива и выбор типа

Из списка: `np.array`

```
a = np.array([1, 2, 3, 4])
print(a)
a = np.array([1., 2., 3., 4.])
print(a)
```

```
[1 2 3 4]
[1. 2. 3. 4.]
```

Нули и единицы

```
print(np.zeros(4))
print(np.ones(4))
```

```
[0. 0. 0. 0.]
[1. 1. 1. 1.]
```

Тип задаётся через `dtype`

```
a = np.zeros(4, dtype="int32")
b = np.ones(4, dtype="bool")
print(a)
print(b)
```

```
[0 0 0 0]
[ True  True  True  True]
```

У элементов массива общий тип `dtype`.

`zeros` и `ones` по умолчанию создают массив типа `float64`.

Заполнение массива: `a.fill()`

Команды выполняются по порядку

```
a = np.array([1, 2, 3, 4])
a.fill(5)
print(a)
```

```
a.fill(2.5)
print(a)
```

```
a = a.astype("float")
a.fill(2.5)
print(a)
```

Содержимое массива

```
[5 5 5 5]
```

`fill` меняет все элементы на месте.

```
[2 2 2 2]
```

Тип остаётся целым: дробная часть отбрасывается.

```
[2.5 2.5 2.5 2.5]
```

`astype` создаёт массив нового типа.



arange: задаём шаг

```
np.arange(start, stop, step)
```

Начало включается, граница `stop` не включается.

Вызов	Элементы массива
`np.arange(10)`	`0 1 2 3 4 5 6 7 8 9`
`np.arange(1, 10)`	`1 2 3 4 5 6 7 8 9`
`np.arange(1, 10, 2)`	`1 3 5 7 9`
`np.arange(10, 1, -2)`	`10 8 6 4 2`



`linspace`: задаём число точек

```
a, step = np.linspace(1, 10, 10, retstep=True)
print(a)
print(step)
```

```
[ 1.  2.  3.  4.  5.  6.  7.  8.  9. 10.]
1.0
```

По умолчанию обе границы включены

`num=10` — десять точек от 1 до 10.

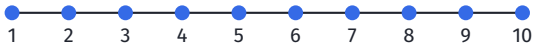
`retstep=True` возвращает также шаг.

Можно указать целый тип

```
np.linspace(1, 10, 10,
            dtype=int)
```

В этом примере: `[1, 2, ..., 10]`.

$$\Delta x = \frac{10-1}{10-1} = 1$$



Массивы из списковых выражений

Линейная последовательность

```
a = np.array([
    1 + i for i in range(10)
])
print(a)
```

```
[ 1  2  3  4  5  6  7  8  9 10]
```

Квадраты чисел

```
a = np.array([
    (1 + i)**2 for i in range(10)
])
```

```
1, 4, 9, 16, 25, 36,
49, 64, 81, 100
```

Двумерный массив: $a_{ij} = ij$

```
a = np.array([
    [i * j for j in range(4)]
    for i in range(3)
])
```

$i \backslash j$	0	1	2	3
$i = 0$	0	0	0	0
$i = 1$	0	1	2	3
$i = 2$	0	2	4	6

Внутренний цикл создаёт строку, внешний — три строки.

Случайные числа: два распределения

Равномерное распределение

```
a = np.random.rand(10)
```

10 чисел из интервала $[0, 1)$.

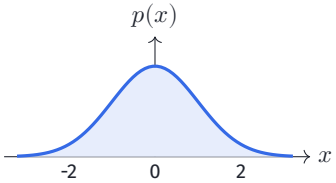


Равные интервалы внутри $[0, 1)$ имеют одинаковую вероятность.

Нормальное распределение

```
b = np.random.randn(10)
d = np.random.normal(0, 1, 10)
```

Оба вызова: $\mu = 0, \sigma = 1$.



В `normal(loc, scale, size)`: `loc` — среднее, `scale` — стандартное отклонение.

Графики показывают плотность распределений. При повторном вызове числа могут измениться.

Случайные целые числа и выбор элементов

Целые числа от 1 до 19

```
c = np.random.randint(1, 20, 10)
print(c)
```

Один из возможных результатов:

```
[ 2  1  5  9  2 10  2 13  3 18]
```



Третий аргумент — число элементов. Верхняя граница 20 исключена.

Выбор из готового массива

```
a = np.array([1, 2, 3, 4, 5,
              6, 7, 8, 9])
print(np.random.choice(a, 3))
```

Один из возможных результатов:

```
[3 8 7]
```



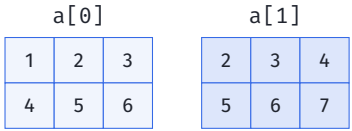
По умолчанию `replace=True`: один элемент может быть выбран несколько раз.

Свойства массива: форма, размер и тип

```
a = np.array([[[1, 2, 3], [4, 5, 6]],
              [[2, 3, 4], [5, 6, 7]]])
```

```
type(a)   numpy.ndarray
a.shape   (2, 2, 3)
a.ndim    3
a.size    12
```

`a.dtype` — тип элементов; в данном окружении `int64`.



2 слоя, 2 строки, 3 столбца

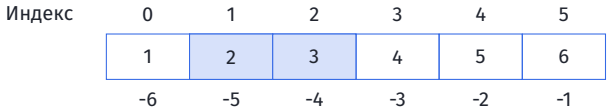
$2 \cdot 2 \cdot 3 = 12$ элементов

Функция `np.shape` работает и со списками

```
a = [[1, 2, 3, 4], [2, 4, 6, 8]]
print(np.shape(a))  # (2, 4)
```

Индексация и срезы: одномерный массив

```
a = np.array([1, 2, 3, 4, 5, 6])
```




```
print(a[1:3])   # [2 3]
print(a[1:-2])  # [2 3 4]
print(a[::2])   # [1 3 5]
```

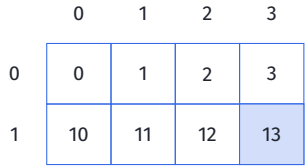
```
a[0] = 10
print(a)
# [10  2  3  4  5  6]
```

`a[start:stop:step]`: индекс `stop` не входит в срез.

Индексация: строки и столбцы

```
a = np.array([[0, 1, 2, 3],
              [10, 11, 12, 13]])
print(a[1, 3])
print(a[1])
print(a[:, 1])
```

```
13
[10 11 12 13]
[ 1 11]
```



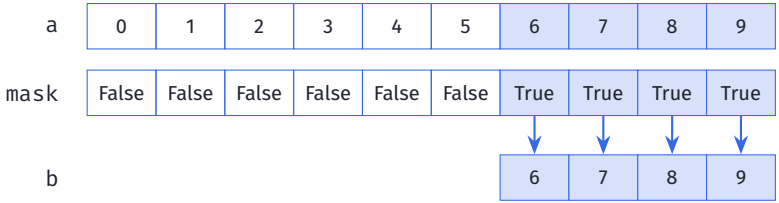
a[строка, столбец]

: выбирает все позиции по оси.

```
a[1, 3] = -1
# [[ 0  1  2  3]
#  [10 11 12 -1]]
```

Логическая маска: выбор по условию

```
a = np.arange(0, 10)
mask = (a > 5)
b = a[mask]
print(b)  # [6 7 8 9]
```



В результат попадают только элементы, которым соответствует `True`.

Срез массива и независимая копия

Обычный срез разделяет данные

```
a = np.array([0, 1, 2, 3, 4])
b = a[2:4]
b[0] = 10
print(a)
```

```
[ 0  1 10  3  4]
```

`copy()` создаёт свои данные

```
a = np.array([0, 1, 2, 3, 4])
b = a[2:4].copy()
b[0] = 10
print(a)
```

```
[0 1 2 3 4]
```



Для списка `a = [1, 2, 3, 4, 5]` срез `b = a[2:4]` создаёт новый список: присваивание `b[0] = 10` не меняет `a`.

Объединение массивов по существующей оси

```
x = np.array([[0, 1, 2], [10, 11, 12]])
y = np.array([[50, 51, 52], [60, 61, 62]])
```

`axis=0`: добавляем строки

```
np.concatenate((x, y))
np.vstack((x, y))
```

0	1	2
10	11	12
50	51	52
60	61	62

$(4, 3)$

`axis=1`: добавляем столбцы

```
np.concatenate((x, y), axis=1)
np.hstack((x, y))
```

0	1	2	50	51	52
10	11	12	60	61	62

$(2, 6)$

В обоих случаях остаются две оси. Размеры по другой оси должны совпадать.

Новая ось: два массива как два слоя

```
x = np.array([[0, 1, 2],
              [10, 11, 12]])
y = np.array([[50, 51, 52],
              [60, 61, 62]])
a = np.array((x, y))
print(a.shape)
```

```
(2, 2, 3)
```

0	1	2
10	11	12

a[0] = x

50	51	52
60	61	62

a[1] = y

$$\underbrace{2}_{\text{слои}} \times \underbrace{2}_{\text{строки}} \times \underbrace{3}_{\text{столбцы}}$$

Изменение формы и транспонирование

`reshape` возвращает массив новой формы

```
a = np.arange(6)
b = a.reshape((2, 3))
print(a)
print(b)
```

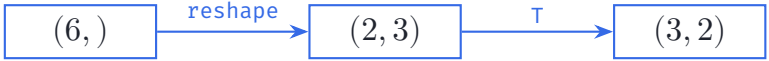
```
[0 1 2 3 4 5]
[[0 1 2]
 [3 4 5]]
```

`T`: строки становятся столбцами

```
print(b.T)
```

```
[[0 3]
 [1 4]
 [2 5]]
```

Присваивание `a.shape = (2, 3)` меняет форму самого `a`.



Сортировка: новый массив или изменение на месте

```
data = np.array([3, 1, 4, 1, 5])
```

np.sort(data)

```
sorted_data = np.sort(data)
print(sorted_data)
print(data)
```

```
[1 1 3 4 5]
[3 1 4 1 5]
```

Исходный массив сохраняется.

data.sort()

```
data.sort()
print(data)
```

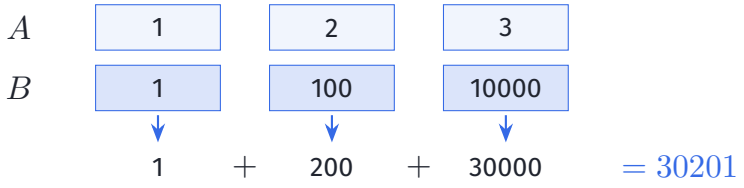
```
[1 1 3 4 5]
```

Меняется сам массив `data`.



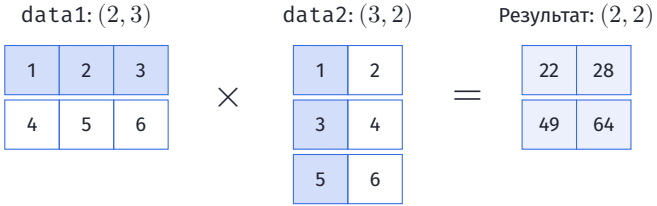
Скалярное произведение векторов

```
A = np.array([1, 2, 3])
B = np.array([1, 100, 10000])
print(np.dot(A, B))  # 30201
```



$A \cdot B = \sum_i A_i B_i$ — результатом является одно число.

Матричное произведение



$$C_{00} = 1 \cdot 1 + 2 \cdot 3 + 3 \cdot 5 = 22$$

```
data1 = np.array([[1, 2, 3], [4, 5, 6]])
data2 = np.array([[1, 2], [3, 4], [5, 6]])
print(np.matmul(data1, data2))
```

Число столбцов первого массива равно числу строк второго.

NumPy и MATLAB: виды умножения

Поэлементное
NumPy: `A * B`
MATLAB: `A .* B`



Матричное
NumPy: `A @ B`
MATLAB: `A * B`



Скалярное
NumPy: `np.dot(a, b)`
MATLAB: `dot(a, b)`



Внешнее
NumPy: `np.outer(a, b)`
MATLAB: `a.' * b`



a, b: в NumPy — одномерные массивы, в MATLAB — строки.

Линейная алгебра: определитель и обратная матрица

```
A = np.array([[2.0, -1.0, 0.0],
              [-1.0, 2.0, -1.0],
              [0.0, -1.0, 2.0]])
```

Определитель

```
print(np.linalg.det(A))
```

```
4.0
```

det $A \neq 0$: обратная матрица существует.

Обратная матрица

```
A_inverse = np.linalg.inv(A)
print(A_inverse)
```

```
[[0.75 0.5  0.25]
 [0.5  1.   0.5 ]
 [0.25 0.5  0.75]]
```

$$AA^{-1} = I = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Собственные значения и векторы

```
A = np.array([[2., -1., 0.],
              [-1., 2., -1.],
              [0., -1., 2.]])
lam, v = np.linalg.eig(A)
print(lam)
print(v)
```

`lam[i]` — собственное значение.

`v[:, i]` — соответствующий столбец собственных векторов.

$$A\mathbf{v}_i = \lambda_i \mathbf{v}_i$$

Собственные значения

$$2+\sqrt{2}, \quad 2, \quad 2-\sqrt{2}$$

$$3.4142, \quad 2.0000, \quad 0.5858$$

Одна собственная пара

$$\lambda = 2, \qquad \mathbf{v} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix}$$

Порядок значений и знаки векторов в выводе могут отличаться.

Решение системы линейных уравнений

```
A = np.array([[2., -1., 0.],
              [-1., 2., -1.],
              [0., -1., 2.]])
b = np.array([2.0, -1.0, 3.0])
x = np.linalg.solve(A, b)
print(x)
```

```
[1.75 1.5  2.25]
```

Система $Ax = b$

$$\begin{cases} 2x_0 - x_1 = 2, \\ -x_0 + 2x_1 - x_2 = -1, \\ -x_1 + 2x_2 = 3. \end{cases}$$

Подстановка решения

$$2 \cdot 1.75 - 1.5 = 2$$

$$-1.75 + 2 \cdot 1.5 - 2.25 = -1$$

$$-1.5 + 2 \cdot 2.25 = 3$$

`solve(A, b)` вычисляет решение напрямую; заранее находить `inv(A)` не требуется.

4

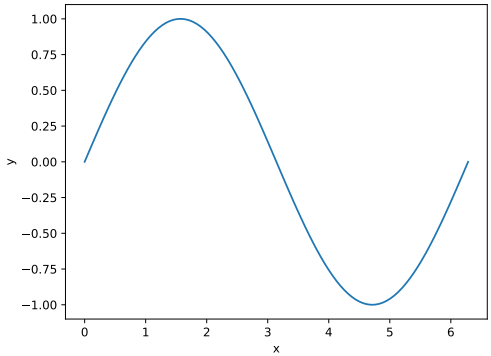
Визуализация с Matplotlib

Первый график в Matplotlib

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2 * np.pi, 100)
y = np.sin(x)

plt.plot(x, y)
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```



`plot(x, y)` соединяет точки линией.

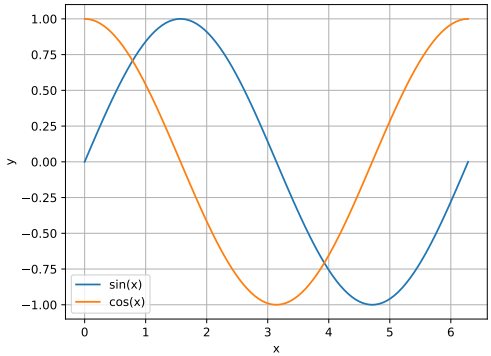
Массивы x и y содержат по 100 значений.

Несколько кривых и легенда

```
x = np.linspace(0, 2 * np.pi, 100)
y1 = np.sin(x)
y2 = np.cos(x)

plt.plot(x, y1, label='sin(x)')
plt.plot(x, y2, label='cos(x)')

plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid()
plt.show()
```



`label` задаёт подпись кривой.

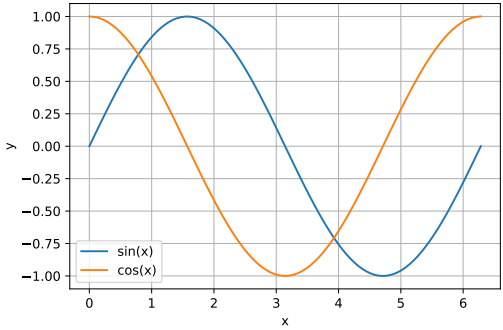
`legend()` показывает легенду.

Размер рисунка и сохранение

```
plt.figure(figsize=(6, 4), dpi=100,
           facecolor='white')

plt.plot(x, y1, label='sin(x)')
plt.plot(x, y2, label='cos(x)')

plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid()
plt.savefig('plot.png', dpi=300,
            bbox_inches='tight')
plt.show()
```



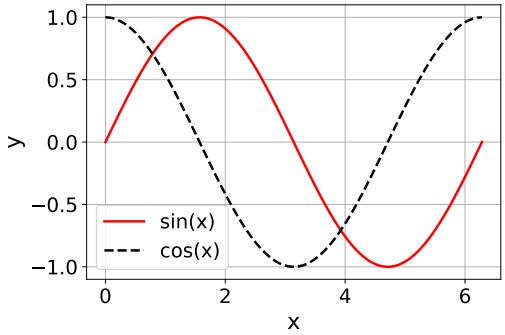
`figsize=(6, 4)` — размер в дюймах.

При `dpi=100`: 600 × 400 пикселей.

При сохранении с `dpi=300`: 1800 × 1200 до обрезки полей.

Цвет, линии и подписи

```
plt.figure(figsize=(6, 4), dpi=100,
           facecolor='white')
plt.plot(x, y1, label='sin(x)',
         color='red', linewidth=2)
plt.plot(x, y2, label='cos(x)',
         color='black', linestyle='--',
         linewidth=2)
plt.xlabel('x', fontsize=18)
plt.ylabel('y', fontsize=18)
plt.tick_params(labelsize=16)
plt.legend(fontsize=16)
plt.grid()
plt.savefig("plot.png", dpi=300,
            bbox_inches="tight")
plt.show()
```



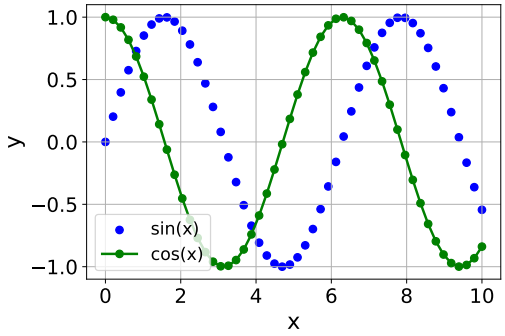
`linewidth` — толщина линии.

`fontsize` — размер текста.

`tick_params` — подписи делений.

Точки и линии с маркерами

```
x = np.linspace(0, 10, 50)
y1 = np.sin(x)
y2 = np.cos(x)
plt.figure(figsize=(6, 4), dpi=100)
plt.scatter(x, y1, color='blue',
            label='sin(x)')
plt.plot(x, y2, color='green',
         marker='o', linestyle='-',
         linewidth=2, label='cos(x)')
plt.xlabel('x', fontsize=18)
plt.ylabel('y', fontsize=18)
plt.tick_params(labelsize=16)
plt.legend(fontsize=14)
plt.grid()
plt.show()
```

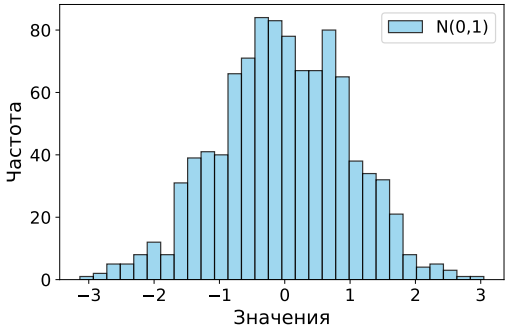


`scatter` показывает отдельные точки.

`plot(..., marker='o')` добавляет маркеры к линии.

Гистограмма

```
data = np.random.normal(
    loc=0, scale=1, size=1000)
plt.figure(figsize=(6, 4), dpi=100)
plt.hist(data, bins=30,
    color='skyblue',
    edgecolor='black',
    alpha=0.8, label='N(0,1)')
plt.xlabel('Значения', fontsize=16)
plt.ylabel('Частота', fontsize=16)
plt.tick_params(labelsize=14)
plt.legend(fontsize=14)
plt.tight_layout()
plt.show()
```



`bins=30` — 30 интервалов.

Высота столбца — число значений в интервале.

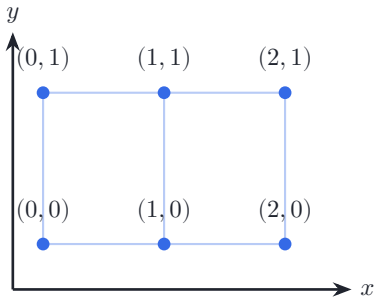
Координатная сетка: `meshgrid`

```
x = np.array([0, 1, 2])
y = np.array([0, 1])
X, Y = np.meshgrid(x, y)
```

Координаты узлов

$$X = \begin{pmatrix} 0 & 1 & 2 \\ 0 & 1 & 2 \end{pmatrix} \qquad Y = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix}$$

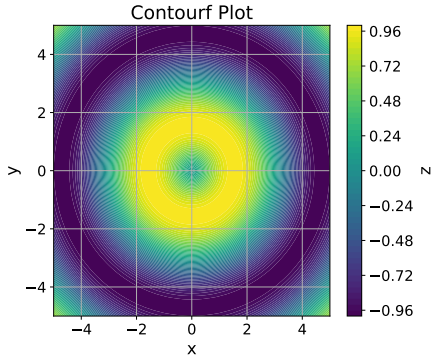
X хранит координаты по горизонтали, Y — по вертикали.



Три значения x и два значения y: шесть узлов сетки.

Двумерное поле: contourf

```
x = np.linspace(-5, 5, 100)
y = np.linspace(-5, 5, 100)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))
plt.figure(figsize=(6, 5))
contour = plt.contourf(
    X, Y, Z, levels=50,
    cmap='viridis')
cbar = plt.colorbar(contour)
cbar.set_label('z', fontsize=16)
plt.xlabel('x', fontsize=16)
plt.ylabel('y', fontsize=16)
plt.title('Contourf Plot', fontsize=18)
plt.tick_params(labelsize=14)
cbar.ax.tick_params(labelsize=14)
plt.grid(True)
plt.tight_layout()
plt.show()
```

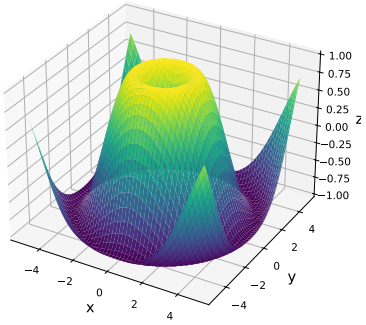


Цвет показывает значение Z.

`levels` управляет числом цветовых уровней.

Трёхмерная поверхность

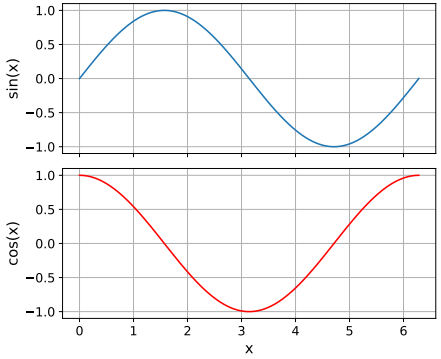
```
x = np.linspace(-5, 5, 100)
y = np.linspace(-5, 5, 100)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))
fig = plt.figure(figsize=(7, 5))
ax = fig.add_subplot(111,
    projection='3d')
surf = ax.plot_surface(X, Y, Z,
    cmap='viridis')
ax.set_xlabel('x', fontsize=14)
ax.set_ylabel('y', fontsize=14)
ax.set_zlabel('z', fontsize=14)
plt.tight_layout()
plt.show()
```



Те же X, Y, Z: значение поля теперь задаёт высоту поверхности.

Несколько графиков: subplots

```
x = np.linspace(0, 2 * np.pi, 100)
fig, axes = plt.subplots(2, 1,
    figsize=(6, 5), sharex=True)
axes[0].plot(x, np.sin(x))
axes[0].set_ylabel('sin(x)',
                   fontsize=14)
axes[1].plot(x, np.cos(x), color='red')
axes[1].set_ylabel('cos(x)',
                   fontsize=14)
axes[1].set_xlabel('x', fontsize=14)
for ax in axes:
    ax.tick_params(labelsize=12)
    ax.grid()
fig.tight_layout()
plt.show()
```



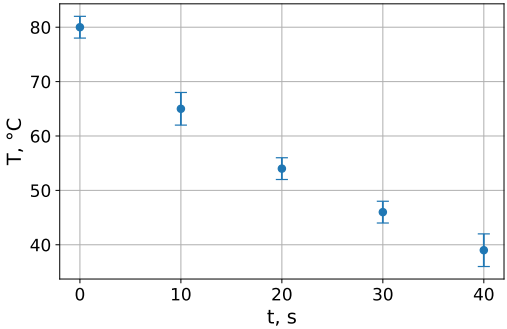
`fig` — весь рисунок.

`axes[0]` и `axes[1]` — два графика с общей осью x.

Погрешности: errorbar

Учебный пример: температура и погрешность измерения.

```
t = np.array([0, 10, 20, 30, 40])
T = np.array([80, 65, 54, 46, 39])
dT = np.array([2, 3, 2, 2, 3])
plt.figure(figsize=(6, 4))
plt.errorbar(t, T, yerr=dT,
             fmt='o', capsize=5)
plt.xlabel('t, s', fontsize=16)
plt.ylabel('T, °C', fontsize=16)
plt.tick_params(labelsize=14)
plt.grid()
plt.tight_layout()
plt.show()
```

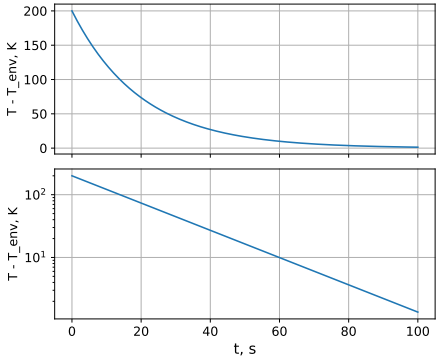


`yerr=dT` задаёт погрешности по вертикали.

Каждый отрезок идёт от `T-dT` до `T+dT`.

Логарифмическая шкала: semilogy

```
t = np.linspace(0, 100, 200)
dT = 200 * np.exp(-0.05 * t)
fig, axes = plt.subplots(2, 1,
    figsize=(6, 5), sharex=True)
axes[0].plot(t, dT)
axes[1].semilogy(t, dT)
for ax in axes:
    ax.set_ylabel('T - T_env, K',
                  fontsize=12)
    ax.tick_params(labelsize=12)
    ax.grid()
axes[1].set_xlabel('t, s', fontsize=14)
fig.tight_layout()
plt.show()
```



На логарифмической оси y экспонента выглядит как прямая. Значения по этой оси должны быть положительными.

Сохранение и загрузка численных данных

Текстовый формат: `txt`

```
x = np.linspace(0, 1, 11)
y = x**2
data = np.column_stack((x, y))
np.savetxt("data.txt", data)
data = np.loadtxt("data.txt")
print(data[:3])
```

```
[[0.   0.  ]
 [0.1  0.01]
 [0.2  0.04]]
```

Формат NumPy: `npy`

```
rng = np.random.default_rng(1)
A = rng.random((100, 100))
np.save("A.npy", A)
B = np.load("A.npy")
print(B.shape)
print(np.array_equal(A, B))
```

```
(100, 100)
True
```

Итоги лекции

Python

Типы данных, условия, циклы и функции.

NumPy

Массивы, индексация, поэлементные операции и линейная алгебра.

Matplotlib

Построение графиков, оформление осей и сохранение рисунков.

GitHub

`github.com/lyushisyan/computational-methods-course`